



RELASI KELAS

Pemrograman Berorientasi Objek

Politeknik Negeri Malang

Overview, Agenda & Learning Outcomes

Setelah sesi teori ini, mahasiswa mampu memahami relasi Uses-A / Has-A (Composition & Aggregation), delegasi antar objek, array of object, dan menerapkan defensive programming pada relasi antar class.

BAGIAN 1 — Konsep & Fondasi (50 menit)

Relasi Objek, Tiga Jenis Relasi, Composition vs Aggregation, Memory Model

BAGIAN 2 — Studi Kasus / Live Coding (110 menit)

Laptop→Processor (+Printer/Uses-A), Karakter→Senjata, Mahasiswa→Jurusan

BAGIAN 3 — Error Handling & Desain (40 menit)

Pitfalls, Defensive Programming, Heuristik Desain Relasi

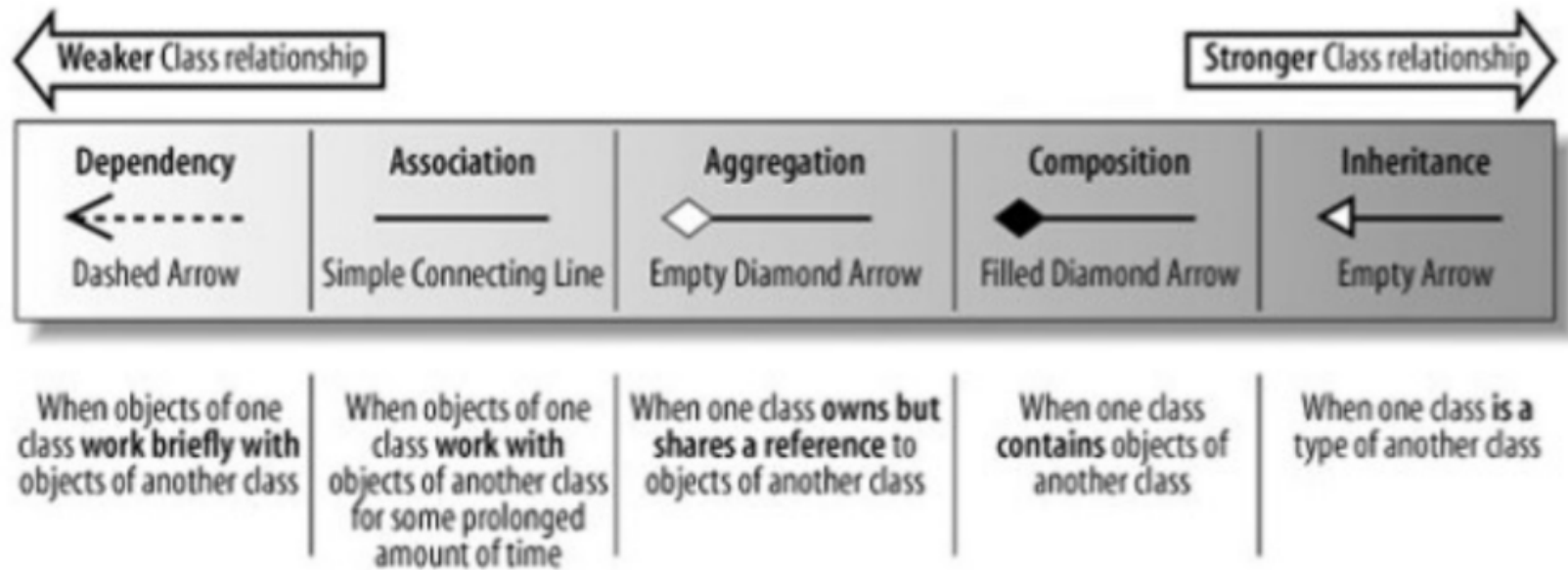
PENUTUP — Ringkasan, Preview Praktikum, Tanya Jawab (20 menit)

Relasi Class

Jenis-jenis Relasi Class :

- ✓ Dependence (*uses-a*)
- ✓ Aggregation (*has-a*)
- ✓ Inheritance (*is-a*)

(Horstmann dan Cornell, 2008)



Relasi Objek: Konsep Dasar

- Sejauh ini, atribut suatu class yang telah dibuat bertipe sederhana (primitive), seperti double, int, float dll
- Class bisa memiliki atribut yang bertipe class lain, dengan kata lain atribut itu berupa objek dari class lain (bertipe objek)
- Sebuah Library bisa memiliki banyak Book sebagai atributnya, sebuah Car bisa memiliki Engine sebagai atributnya
- Dalam kasus yang lebih kompleks, dalam suatu sistem akan ditemukan lebih dari satu class yang saling memiliki keterkaitan antara class satu dengan yang lain.
- Jika pada percobaan-percobaan sebelumnya, mayoritas kasus yang sudah dikerjakan hanya fokus pada satu class saja, pada materi ini akan dibahas contoh kasus yang melibatkan beberapa class yang saling berelasi.

Relasi Objek: Konsep Dasar

- Objek di dunia nyata **jarang berdiri sendiri** ? sebuah benda biasanya tersusun dari, atau membutuhkan, benda lain.
- Dalam OOP, ini dimodelkan dengan: atribut sebuah class boleh bertipe class lain ? tidak harus tipe dasar seperti int atau String.
- Contoh:
 - class PesawatTerbang, di dalamnya terdapat object Mesin, Sayap, Sirip, dan Pilot.
 - class Laptop yang di dalamnya terdapat Processor.
- Supaya sebuah objek bisa **"memiliki"** objek lain, objek yang dimiliki itu harus sudah dibuat lebih dulu ? baru kemudian dikaitkan/dipasangkan (nanti dilihat caranya lewat *constructor* atau *setter*).

→ Konsep ini yang selanjutnya disebut relasi HAS-A (**Agregasi**): sebuah class "memiliki" object dari class lain sebagai atributnya.

Mengapa Relasi Objek Penting?

✗ Tanpa desain relasi yang baik:

- Semua atribut ditumpuk di satu class → sulit dipelihara
- Struktur data tidak jelas → sulit dikembangkan tim

✓ Dengan desain relasi yang baik:

- Setiap class punya tanggung jawab spesifik (single responsibility)
- Kode lebih reusable, mudah diuji, dan mudah dikembangkan

Relasi antar objek adalah fondasi dari desain sistem berorientasi objek yang baik ☑ dipakai di hampir semua aplikasi nyata.

Tiga Jenis Relasi Antar Class

1. **USES-A** (*Dependency*) ? relasi sementara/lemah
 - Object lain hanya dipakai sesaat, **TIDAK disimpan sebagai atribut**
 - Contoh: method `Driver.mengemudikan(Mobil m)`
2. **HAS-A** (*Association / Composition / Aggregation*)
 - Object lain **DISIMPAN sebagai atribut** (*field*) tetap
 - Contoh: Laptop HAS-A Processor
3. **IS-A** (*Inheritance*) ? akan dibahas mendalam di pertemuan lain

Contoh: Dog IS-A Animal

? Cara cepat membedakan di kode sendiri:

Object itu disimpan sebagai private field di class? → HAS-A. Object itu hanya muncul sebagai parameter method dan tidak disimpan? → USES-A.

Contoh kontras dengan class yang SAMA: `Driver.mengemudikan(Mobil m)` → USES-A (parameter). Tapi jika `Driver` punya private `Mobil mobilSaatIni`; → itu sudah HAS-A (Association) — walau **BUKAN whole-part** seperti *Composition/Aggregation*.

Spektrum Kekuatan Relasi HAS-A

ASSOCIATION (Paling Lemah)

Sekadar "mengenal & menyimpan" referensi
Bukan whole-part, tanpa ownership khusus
Part berdiri sendiri, tidak terikat

Contoh:

Driver — Mobil
(Driver simpan referensi Mobil, tapi Mobil bukan "bagian dari" Driver)

Kode:

```
private Mobil mobilSaatIni;
```

AGGREGATION ◇ (Weak Ownership)

Whole-part, tapi part independen
Jika "whole" dihapus, "part" tetap ada
Part bisa dibagi/pindah ke whole lain

Contoh:

Mahasiswa ◇ Jurusan
Laptop ◇ Processor (di-inject lewat setCPU — lihat Studi Kasus 1)

Kode:

```
Mahasiswa(..., Jurusan jur){  
    Jur = jur;  
}
```

COMPOSITION ◆ (Strong Ownership)

Whole-part, lifecycle part terikat whole
Jika "whole" dihapus, "part" ikut hilang
Part biasanya tidak dibagi

Contoh:

Car ◆ Engine
(Engine di-new LANGSUNG di dalam constructor Car, "mati" bersama Car)

Kode:

```
Engine mesin = new Engine();
```

Ketiganya sama-sama HAS-A (field tersimpan) ☞ bedanya hanya SEBERAPA KUAT ownership-nya.

Karakteristik di Java: Ciri Kode Tiap Relasi

Jenis Relasi	Hubungan Logis	Ciri Kode di Java	Kekuatan
Dependency	Uses-a (memakai sementara)	Muncul sebagai parameter atau variabel lokal di dalam method	1/4 (Paling Lemah)
Association	Peer-to-peer (saling tahu)	Field menyimpan REFERENSI objek lain (disimpan, bukan dibuat)	2/4
Aggregation ◇	Has-a (memiliki secara longgar)	Objek "part" dibuat DI LUAR, lalu di-inject lewat constructor/setter	3/4
Composition ◆	Part-of (bagian mutlak)	Objek "part" di-new LANGSUNG di dalam class induk (biasanya di constructor)	4/4 (Paling Kuat)

Contoh Composition Sungguhan: Car ♦ Engine

```
public class Engine
{
    private String tipe;

    public Engine()
    {
        this.tipe = "4-silinder";
    }
    public String getTipe()
    {
        return tipe;
    }
}

public class Car
{
    private String merek;
    private Engine mesin;

    public Car(String merek)
    {
        this.merek = merek;
        this.mesin = new Engine();
    }
    public void tampilkanInfo()
    {
        System.out.println("Car: " + merek);
        System.out.println("Engine: " + mesin.getTipe());
    }
}
```

Engine di-new LANGSUNG di dalam constructor Car
❓ bukan diterima dari luar sebagai parameter.

Tidak ada setEngine()/setMesin(). Sekali Car dibuat,
Engine-nya tidak bisa diganti atau diambil dari luar.

✓ Inilah yang menegaskan Composition secara KODE:
lifecycle Engine sepenuhnya dikendalikan oleh Car.

Bandingkan dengan Laptop ◇ Processor (Aggregation):
Processor dibuat DI LUAR, lalu di-inject via setCPU().

Model Memori: Stack vs Heap



PENTING: `lap.setCPU(cpu)` menyalin REFERENCE, bukan menggandakan objek Processor.

Berlaku juga saat objek dilewatkan sebagai parameter method (mis. `cetakDokumen(Printer p)`): yang disalin adalah REFERENCE-nya. Mengubah isi objek di dalam method akan terlihat di luar; tapi mengganti variabel parameter ke objek baru TIDAK memengaruhi variabel asal pemanggil.

*Kumpulan objek yang saling terhubung lewat reference seperti ini disebut **object graph** — istilah yang akan sering muncul saat mendiskusikan relasi antar objek (mis. `Laptop → Processor`, atau `Mahasiswa → Jurusan` yang dipakai bersama).*

Studi Kasus 1: Laptop ◇ Processor (+ Uses-A Printer)

Laptop
- Merek: String
- CPU: Processor
+ setMerek(mrk: String)
+ getMerek(): String
+ setCPU(proc: Processor)
+ getCPU(): Processor
+ tampilkanInfo()
+ cetakDokumen(p: Printer, namaFile: String)

1 ◇ — HAS-A — 1
(proc dibuat di luar, di-inject
lewat setCPU)

Processor
- Merek: String
- Frekuensi: double
+ setMerek(mrk: String)
+ getMerek(): String
+ setFrekuensi(frek: double)
+ getFrekuensi(): double
+ tampilkanInfo()

--- USES-A (parameter) --->

Printer
- Merek: String
+ cetak(namaFile: String)

Diamond ◇ = HAS-A/Aggregation (field, di-inject dari luar). Garis putus-putus = USES-A (parameter saja, tidak disimpan) ? akan kita buktikan di kode nanti.

Implementasi: class Processor

```
public class Processor
{
    private String Merek;
    private double Frekuensi;

    public String getMerek()
    {
        return Merek;
    }
    public void setMerek(String mrk)
    {
        Merek = mrk;
    }
    public double getFrekuensi()
    {
        return Frekuensi;
    }
    public void setFrekuensi(double frek)
    {
        Frekuensi = frek;
    }
    public void tampilkanInfo()
    {
        System.out.println("Merek processor: " + Merek);
        System.out.println("Frekuensi processor: " +
            Frekuensi + " GHz");
    }
}
```

- Class Processor berdiri sendiri.
- Memiliki atribut privat Merek dan Frekuensi.
- Disediakan getter/setter dan method tampilkanInfo() untuk menampilkan datanya sendiri.

Implementasi: class Laptop (HAS-A Processor)

```
public class Laptop
{
    private String Merek;
    private Processor CPU;

    public String getMerek()
    {
        return Merek;
    }
    public void setMerek(String mrk)
    {
        Merek = mrk;
    }
    public Processor getCPU()
    {
        return CPU;
    }
    public void setCPU(Processor proc)
    {
        CPU = proc;
    }
    public void tampilkanInfo()
    {
        System.out.println("Merek laptop: " + Merek);
        CPU.tampilkanInfo();
    }
}
```

- Perhatikan atribut:
private Processor CPU;
- Inilah relasi HAS-A: Laptop "memiliki" Processor sebagai bagian dari dirinya.
- setCPU(proc) menerima REFERENCE ke objek Processor yang sudah dibuat sebelumnya.

✓ Mengapa Aggregation, bukan Composition

Processor dibuat DI LUAR class Laptop, lalu di-inject lewat setCPU() — bukan di-new langsung di dalam Laptop.

Menjalankan: TestLaptop

```
public class TestLaptop
{
    public static void main(String[] args)
    {
        Processor proc = new Processor();
        proc.setMerek("Intel Core i7");
        proc.setFrekuensi(2400);

        Laptop lap = new Laptop();
        lap.setMerek("Asus");
        lap.setCPU(proc);

        lap.tampilkanInfo();
    }
}
```

Output program

```
G:\Java>java TestLaptop
```

```
Merek laptop: Asus
Merek processor: Intel Core i7
Frekuensi processor: 2400.0 GHz
```

Langkah:

1. Buat objek Processor, isi datanya
2. Buat objek Laptop
3. Hubungkan: `lap.setCPU(proc)`
4. `lap.tampilkanInfo()` otomatis memanggil `CPU.tampilkanInfo()` → delegasi antar objek

Mengakses Atribut Bertingkat (Chaining)

```
public class TestLaptop
{
    public static void main(String[] args)
    {
        Processor proc = new Processor();
        proc.setMerek("Intel Core i7");
        proc.setFrekuensi(2400);
        Laptop lap = new Laptop();
        lap.setMerek("Asus");
        lap.setCPU(proc);
        System.out.println("Laptop Merek " + lap.getMerek());
        System.out.println("dengan processor " + lap.getCPU().getMerek());
    }
}
```

⚠ Dua risiko dari `lap.getCPU().getMerek()`:

1. Jika `getCPU()` mengembalikan null, baris ini CRASH (`NullPointerException`).
2. Kode LUAR bisa menembus ke CPU dan mengubahnya langsung, misalnya: `lap.getCPU().setFrekuensi(9999);` ← *Laptop tidak pernah tahu ini terjadi!*

Ini melanggar *Law of Demeter* ("jangan bicara dengan orang asing") dan berisiko *encapsulation leak*. Solusinya ada di slide berikutnya.

Mengatasi Risiko #1: NullPointerException

✗ MASALAH (akan crash):

```
Laptop lap = new Laptop();  
lap.setMerek("Dell");  
// CPU tidak pernah di-set (masih null)  
lap.tampilkanInfo(); // CRASH! CPU.tampilkanInfo()
```

✓ SOLUSI (null check di dalam tampilkanInfo):

```
public void tampilkanInfo() {  
    System.out.println("Merek laptop: " + Merek);  
    if (CPU != null) {  
        CPU.tampilkanInfo();  
    } else {  
        System.out.println("Processor: belum terpasang");  
    }  
}
```

Mengatasi Risiko #2: Tell, Don't Ask

Laptop
- Merek: String
- CPU: Processor
+ setMerek/getMerek
+ setCPU/getCPU
+ tampilkanInfo()
+ overclock(): void ← BARU

Processor
- Merek: String
- Frekuensi: double
+ setFrekuensi/getFrekuensi
+ tampilkanInfo()



Prinsip: TELL, DON'T ASK

Daripada kode luar "bertanya" (`getCPU()`) lalu mengubah sendiri, lebih baik Laptop diberi "perintah" (`overclock()`) dan Laptop yang mengurus detail internalnya:

```
public void overclock() { CPU.setFrekuensi(CPU.getFrekuensi() + 100); }
```

Catatan penting: bukan berarti chaining SELALU buruk. Laptop BOLEH memanggil `CPU.setFrekuensi()` di dalam method-nya sendiri ☑ itu tetap di dalam batas class-nya.

Yang perlu dihindari: KODE DI LUAR class Laptop yang menembus ke CPU secara langsung.

Uses-A dalam Kode: Laptop Memakai Printer

Printer BUKAN atribut Laptop — hanya lewat sebagai parameter method → USES-A.

```
public class Printer {  
    private String Merek;  
    public Printer(String merek) { Merek = merek; }  
    public void cetak(String namaFile) {  
        System.out.println("Mencetak " + namaFile + " di " + Merek);  
    }  
}  
  
// Method baru di class Laptop – TIDAK ada field Printer!  
public void cetakDokumen(Printer p, String namaFile) {  
    p.cetak(namaFile);  
}  
  
// tambahan di class TestLaptop – buat objek printer dan passing saat memanggil cetakDokumen  
Printer prn = new Printer("Epson L120");  
lap.cetakDokumen(prn, "laporan.pdf");  
// Output: Mencetak laporan.pdf di Epson L120
```

Catatan desain: jika Laptop SERING mencetak ke printer yang SAMA, pertimbangkan simpan sebagai field (Has-A). Uses-A cocok untuk pemakaian insidental saja.

Studi Kasus 2: Karakter ◇ Senjata[] (Array of Object)

Karakter
- Nama: String
- SenjataTerpasang: Senjata[]
+ getNama(): String
+ setNama(String nam)
+ getSenjataTerpasang(): Senjata[]
+ setSenjataTerpasang(Senjata[] sjt)
+ tampilkanInfo()

1 ◇ — HAS-A — 1..* (satu atau lebih)

Senjata
- Nama: String
- Kekuatan: int
+ Senjata(nama: String, kekuatan: int)
+ getNama(): String
+ setNama(String nam)
+ getKekuatan(): int
+ setKekuatan(kek: int)
+ tampilkanInfo()

Atribut objek bisa lebih dari satu → gunakan array untuk menyimpannya. Karakter memiliki banyak Senjata (aggregation ? dibuat di luar, di-inject lewat setter).

Implementasi: class Senjata

```
public class Senjata
{
    private String Nama;
    private int Kekuatan;

    public Senjata(String nam, int kek)
    {
        Nama = nam;
        Kekuatan = kek;
    }
    public String getNama()
    {
        return Nama;
    }
    public void setNama(String nm)
    {
        Nama = nm;
    }
    public int getKekuatan()
    {
        return Kekuatan;
    }
    public void setKekuatan(int k)
    {
        Kekuatan = k;
    }
}
```

Class Senjata mirip Processor: berdiri sendiri, menyimpan Nama dan Kekuatan.

Constructor Senjata(nam, kek) memudahkan pembuatan objek langsung dengan data awal terisi
❓ tidak perlu setNama()/setKekuatan() terpisah.

Implementasi: class Karakter (HAS-A Senjata[])

```
public class Karakter
{
    private String Nama;
    private Senjata SenjataTerpasang[];

    public String getNama()
    {
        return Nama;
    }
    public void setNama(String nam)
    {
        Nama = nam;
    }
    public Senjata[] getSenjataTerpasang()
    {
        return SenjataTerpasang;
    }
    public void setSenjataTerpasang(Senjata[] sjt)
    {
        SenjataTerpasang = sjt;
    }
    public void tampilkanInfo()
    {
        System.out.println("\nNama karakter: " + Nama);
        for (int i = 0; i < SenjataTerpasang.length; i++)
        {
            System.out.println("Senjata " + i + ": " +
                               SenjataTerpasang[i].getNama());
        }
    }
}
```

`private Senjata SenjataTerpasang[];` → array of object: tiap elemen REFERENCE ke objek Senjata yang berbeda-beda.

💡 Tanpa override `toString()`, print objek langsung (mis. `print(senjata)`) akan tampil `"Senjata@1b6d358"` ❓ bukan info berguna. Karena itu dipakai method `tampilkanInfo()` sendiri sebagai ganti.

Menjalankan: TestKarakter

```
public class TestKarakter
{
    public static void main(String[] args)
    {
        Senjata snj1 = new Senjata("Pedang", 100);
        Senjata snj2 = new Senjata("Panah", 70);
        Senjata snj3 = new Senjata("Pisau", 30);
        Karakter kar1 = new Karakter();
        kar1.setNama("Arjuna");
        kar1.setSenjataTerpasang(
            new Senjata[]{snj1, snj2} );
        Karakter kar2 = new Karakter();
        kar2.setNama("Kurawa");
        kar2.setSenjataTerpasang( new Senjata[]{snj3} );
        kar1.tampilkanInfo();
        kar2.tampilkanInfo();
    }
}
```

Output program

G:\Java>java TestKarakter

Nama karakter: Arjuna
Senjata 0: Pedang
Senjata 1: Panah

Nama karakter: Kurawa
Senjata 0: Pisau

Setiap objek Karakter memiliki array Senjata sendiri ? Arjuna dan Kurawa tidak saling berbagi senjata yang sama.

`new Senjata[]{snj1, snj2}` membuat array literal berisi REFERENCE ke objek yang sudah dibuat sebelumnya.

Pitfall: Array Index & Elemen Belum Diisi

✗ `ArrayIndexOutOfBoundsException`:

```
Senjata[] snj = new Senjata[3]; // kapasitas 3
snj[5] = new Senjata("Panah", 70); // CRASH!
```

✗ Elemen array belum diisi (null):

```
Senjata[] snj = new Senjata[3];
snj[0] = new Senjata("Pedang", 100);
for (int i = 0; i < snj.length; i++) {
    snj[i].tampilkanInfo(); // CRASH di i=1! (masih null)
}
```

✓ SOLUSI: validasi indeks & null check sebelum akses

```
if (i < snj.length && snj[i] != null) {
    snj[i].tampilkanInfo();
}
```

Pola Defensive Programming (Rangkuman)

POLA 1 ? Null Checking

```
if (obj != null) { obj.method(); }
```

POLA 2 ? Validasi Batas Array

```
if (index >= 0 && index < array.length) { ... }
```

POLA 3 ? Iterasi Aman dengan Counter

```
for (int i = 0; i < jumlahTerisi; i++) { ... }
```

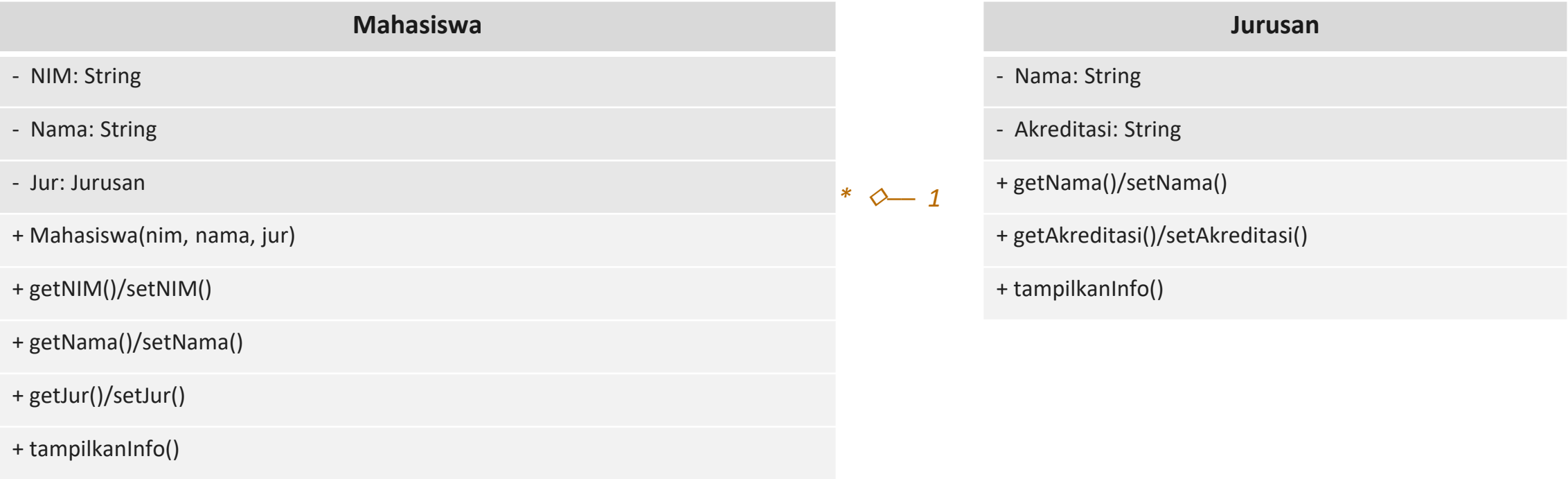
POLA 4 ? Defensive Getter

```
if (CPU == null) throw new IllegalStateException("CPU belum diset");
```



Prinsip: jangan pernah asumsikan data selalu valid ? selalu validasi!

Studi Kasus 3: Mahasiswa ◇ Jurusan (Aggregation)



Berbeda dari Laptop–Processor: Jurusan dibuat TERPISAH lalu "dipinjamkan" ke Mahasiswa lewat constructor. Jurusan tetap ada walau satu Mahasiswa dihapus, dan bisa dipakai bersama banyak Mahasiswa lain (multiplicity * pada sisi Mahasiswa, 1 pada sisi Jurusan).

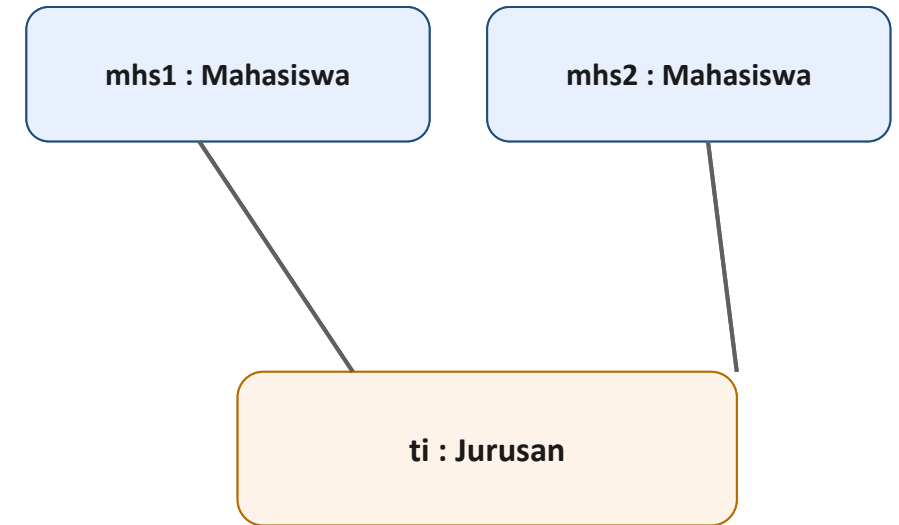
Implementasi: class Jurusan & Mahasiswa

```
public class Jurusan {  
    private String Nama;  
    private String Akreditasi;  
  
    public void setName(String n) {  
        Nama = n;  
    }  
    public String getName() {  
        return Nama;  
    }  
    public void setAkreditasi(String a) {  
        Akreditasi = a;  
    }  
    public String getAkreditasi() {  
        return Akreditasi;  
    }  
}
```

```
public class Mahasiswa {  
    private String NIM, Nama;  
    private Jurusan Jur; // Has-A (Aggregation)  
  
    public Mahasiswa(String nim, String nama,  
                      Jurusan jur) {  
        NIM = nim; Nama = nama; Jur = jur;  
    }  
  
    public void tampilkanInfo() {  
        System.out.println("NIM: " + NIM);  
        System.out.println("Nama: " + Nama);  
        System.out.println("Jurusan: " + Jur.getName());  
    }  
}
```

Membuktikan Aggregation: Berbagi Satu Objek

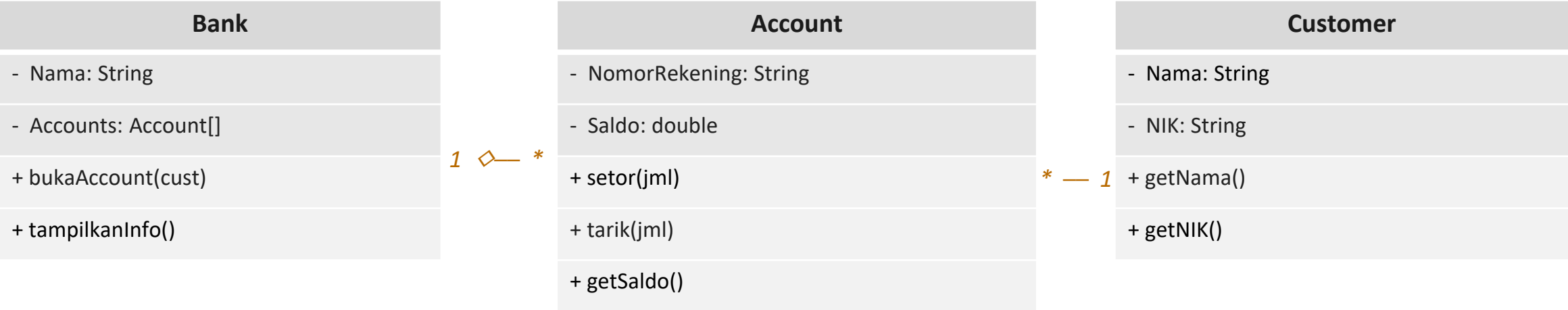
```
Jurusan ti = new Jurusan();  
ti.setNama("Teknologi Informasi");  
ti.setAkreditasi("A");  
  
Mahasiswa mhs1 = new Mahasiswa("111001", "Jones", ti);  
Mahasiswa mhs2 = new Mahasiswa("111002", "Alice", ti);  
// perhatikan: ti yang SAMA dipakai keduanya  
  
mhs1.tampilkanInfo();  
mhs2.tampilkanInfo();  
// Output:  
// NIM: 111001 / Nama: Jones / Jurusan: Teknologi  
Informasi  
// NIM: 111002 / Nama: Alice / Jurusan: Teknologi  
Informasi
```



Objek Jurusan yang SAMA dipakai 2 Mahasiswa

- mhs1 & mhs2 menunjuk ke OBJEK ti yang sama, bukan dua salinan.
- Jika mhs1 dihapus, objek ti TETAP ADA (masih dipakai mhs2).
- Beda dengan Composition: Engine pada Car dibuat & "mati" bersama Car-nya sendiri — tidak didesain untuk dipakai bersama Car lain.

Diskusi: Bank ◇ Account ◇ Customer



? Pertanyaan diskusi (jawab dengan pola Mahasiswa ◇ Jurusan tadi):

1. Bank ◇ Account: Aggregation atau Composition? Jika Bank tempat Account dibuka TUTUP, apakah Account pemilik lain ikut hilang? (petunjuk: biasanya di-migrasi, bukan hilang)
2. Satu Customer bisa punya LEBIH DARI SATU Account (tabungan + giro). Apakah ini mengubah jenis relasi Account—Customer, atau hanya mengubah multiplicity-nya?
3. Bandingkan dengan Laptop ◆ Processor: mengapa Account—Customer BUKAN Composition meskipun sama-sama "dimiliki"?

→ Jawaban singkat: keduanya Aggregation — Account tetap ada lepas dari satu Bank/Customer tertentu, hanya multiplicity-nya yang berbeda (1..* pada sisi Account untuk Customer).

Kapan Perlu Relasi? (Heuristik Desain)

Pertanyaan yang bisa dipakai saat merancang class sendiri:

1. Apakah data ini punya identitas/perilaku sendiri yang berdiri sendiri?

Ya → jadikan class terpisah (relasi). Sekadar nilai sederhana (mis. int x, y pada Titik) → cukup atribut biasa, tidak perlu class baru.

2. Berapa banyak instance dibutuhkan, dan apakah jumlahnya tetap?

Jumlah PASTI & kecil (mis. tepat 2 kamera) → atribut terpisah: KameraDepan, KameraBelakang

Jumlah DINAMIS/banyak (mis. senjata karakter) → array/koleksi

3. Objek ini eksklusif milik satu pemilik, atau bisa dipakai bersama?

Eksklusif, lifecycle terikat → Composition (◆)

Bisa dibagi, lifecycle independen → Aggregation (◇)

Domain yang sudah kita lihat: Laptop/Processor, Karakter/Senjata, Mahasiswa/Jurusan, Driver/Mobil, Bank/Account/Customer ☹ pola yang sama, konteks berbeda. Latihlah mengenali polanya, bukan menghafal contohnya.

Ringkasan & Key Takeaways

- ✓ HAS-A (field tersimpan) → Composition (◆, terikat) atau Aggregation (◇, bisa dibagi)
- ✓ USES-A: object hanya lewat sebagai parameter, tidak disimpan sebagai field
- ✓ Heuristik cepat: disimpan sebagai field → HAS-A; hanya parameter → USES-A
- ✓ Variabel objek menyimpan REFERENCE; objek sesungguhnya ada di Heap
- ✓ Tell, Don't Ask: sembunyikan detail internal, hindari chaining dari luar (Law of Demeter)
- ✓ Array of Object menyimpan banyak reference — pastikan semua terisi & tervalidasi
- ✓ Selalu terapkan defensive programming: null check, validasi batas/index
- ✓ Diagram UML + multiplicity membantu merancang relasi sebelum menulis kode

Latihan Mandiri

Sepeda motor terdiri dari atribut yaitu: Merek, Warna, Mesin dan MaxSpeed, dan juga behavior yaitu menambah dan mengurangi kecepatan. Mesin adalah objek terpisah yang terdiri dari Kapasitas dan Tipe Bahan Bakar.

Buat setidaknya 2 kelas yang terkait, setiap kelas memiliki atribut dan metodenya sendiri dan juga metode getter-setter.

- Gambar diagram UML dari kelas-kelas tersebut (sertakan multiplicity).
- Terapkan minimal 1 pola defensive programming yang telah dipelajari.
- Pertimbangkan: apakah desainmu butuh relasi Uses-A juga (misalnya method yang menerima objek lain sebagai parameter tanpa menyimpannya)?

Pertanyaan?

Terima
kasih

